INSIDE STLC ACADEMY

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

Auto-Waiting & Handling Async Guide

Stop fighting timing issues — work with Playwright's auto-waiting.

Introduction

Most flaky tests come from timing. Playwright auto-waits for elements to be actionable and for assertions to pass — if you let it. This guide shows you how to rely on that and avoid manual sleeps.

How Auto-Waiting Works

- Actions (click, fill) wait for the element to be visible, stable and enabled.
- Web-first assertions (expect(locator)) retry until they pass or time out.
- You almost never need manual waits.

Do This, Not That

Avoid	Use instead
page.waitForTimeout(3000)	await expect(locator).toBeVisible()
Manual polling loops	Web-first assertions
Sleep then click	Just click — it auto-waits
waitForSelector then act	Act on the locator directly

When You Do Need to Wait

- await page.waitForURL(...) after a navigation
- await page.waitForResponse(...) for a specific network call
- await expect(...) for the visible result of async work

Worked Example

Instead of waiting 3 seconds for a toast, assert it: await expect(page.getByText('Saved')).toBeVisible(); — Playwright retries until it appears, so the test is both faster and more reliable.

Practical Exercise

Find any waitForTimeout in your tests and replace it with a web-first assertion. Confirm the test still passes and runs faster.

Best Practices

- Assert outcomes instead of sleeping.
- Wait on events (URL, response), not arbitrary time.
- Trust auto-waiting for actions.

Common Mistakes

- Sprinkling waitForTimeout to "fix" flakiness.
- Waiting on fixed times that fail under load.
- Re-implementing waits Playwright already does.

> **INSIDE STLC PRO TIP**
>
> Every fixed waitForTimeout is either too long (slow suite) or too short (flaky). Replace them all with assertions on the actual outcome you are waiting for.